

StockWise

Smart Inventory & Stock Tracking System

Technical Requirements Document

1. Introduction

This document outlines the complete technical requirements for the StockWise inventory management system. It covers functional requirements (what the system must do), non-functional requirements (how the system must perform), and the full technical stack used across the backend, frontend, database, and testing layers.

2. Technology Stack

Layer	Technology	Details
Backend	Java 21 / Spring Boot	Core application framework — REST APIs, Security, JPA, Web
Database	PostgreSQL 15+	Relational database for persistent storage of all entities
Frontend	React.js 18	Component-based UI framework for dynamic dashboard rendering
Styling	Tailwind CSS	Utility-first CSS framework for responsive design
HTTP Client	Axios	Promise-based HTTP client for frontend-backend API integration
Data Visualization	Chart.js	JavaScript charting library for analytics dashboards
Authentication	Spring Security + JWT	Stateless authentication using JSON Web Tokens
ORM	Spring Data JPA / Hibernate	Object-relational mapping for database entity management
Build Tool (BE)	Apache Maven	Dependency management and build automation for backend
Build Tool (FE)	npm / Vite	Package management and bundling for the frontend
Testing (BE)	JUnit 5 + Mockito	Unit and mock testing for backend service layer
Version Control	Git / GitHub	Source code versioning and team collaboration

Project Management	Jira	Sprint planning, task tracking, and backlog management
UML Modeling	Draw.io	Use Case, Class, and ER Diagrams

3. Functional Requirements

The following table defines the functional requirements of StockWise using MoSCoW prioritization.

3.1 Authentication & User Management

ID	Requirement	Description	Priority
FR-01	User Login	The system shall authenticate users with username and password via a secure login form.	Must Have
FR-02	Role-Based Access	The system shall enforce Admin and Staff roles with different access levels.	Must Have
FR-03	JWT Token Auth	The system shall issue JWT tokens on login and validate them on each request.	Must Have
FR-04	Logout	The system shall allow users to securely terminate their session.	Must Have

3.2 Product & Category Management

ID	Requirement	Description	Priority
FR-05	Add Product	Admins shall be able to create new products with name, quantity, price, threshold, and category.	Must Have
FR-06	Edit Product	Admins shall be able to update any field of an existing product.	Must Have
FR-07	Delete Product	Admins shall be able to remove products from the system.	Must Have
FR-08	List Products	All users shall be able to view a paginated and searchable product list.	Must Have
FR-09	Category CRUD	Admins shall be able to create, read, update, and delete product categories.	Must Have
FR-10	Filter by Category	Users shall be able to filter product listings by category.	Should Have

3.3 Inventory & Alert Management

ID	Requirement	Description	Priority
FR-11	Stock Monitoring	The system shall display current stock quantities for all products in real time.	Must Have
FR-12	Threshold Alerts	The system shall automatically generate an alert when product quantity falls below its threshold.	Must Have
FR-13	Alert Notification	Admins shall receive alerts in the notification panel when critical stock is detected.	Must Have
FR-14	Alert History	The system shall store all past alerts with product reference and timestamp.	Should Have

3.4 Analytics & Reporting

ID	Requirement	Description	Priority
FR-15	Stock Dashboard	The system shall display a visual dashboard with stock level summaries.	Must Have
FR-16	Chart Visualization	The system shall render bar, line, and pie charts using Chart.js for inventory data.	Must Have
FR-17	Low Stock Report	The system shall display a filtered list of all products currently below threshold.	Should Have
FR-18	Category Breakdown	The system shall display a category-wise inventory distribution chart.	Should Have

4. Non-Functional Requirements

Category	Requirement	Details
Performance	API Response Time	All REST API endpoints must respond within 500ms under normal load.
Performance	Dashboard Load	The frontend dashboard must fully render within 2 seconds.
Security	JWT Expiry	Access tokens must expire after 1 hour; refresh tokens after 7 days.
Security	Password Hashing	All user passwords must be hashed using BCrypt before storage.
Security	HTTPS	All production traffic must be served over HTTPS.
Usability	Responsive Design	The UI must be fully usable on desktop, tablet, and mobile screen sizes.
Usability	Error Feedback	The system must display meaningful error messages for all failed operations.

Reliability	Uptime	The system must target 99% uptime in production deployments.
Maintainability	Code Documentation	All backend service methods must include Javadoc comments.
Scalability	Modular Architecture	The backend must use a layered architecture (Controller, Service, Repository) for scalability.

5. Database Schema Requirements

The PostgreSQL relational database must implement the following entities and their relationships:

Entity	Key Fields	Relationships
USERS	id (PK), username, password, role	Manages PRODUCTS (one-to-many)
PRODUCTS	id (PK), name, quantity, threshold, price, category_id (FK)	Belongs to CATEGORIES; Triggers ALERTS
CATEGORIES	id (PK), category_name	Contains PRODUCTS (one-to-many)
ALERTS	id (PK), message, created_at	Triggered by PRODUCTS when quantity < threshold

6. API Endpoint Requirements

The backend RESTful API must expose the following endpoint groups:

- POST /api/auth/login — Authenticate user and return JWT token
- GET /api/products — Retrieve all products (paginated)
- POST /api/products — Create a new product (Admin only)
- PUT /api/products/{id} — Update product details (Admin only)
- DELETE /api/products/{id} — Delete a product (Admin only)
- GET /api/categories — Retrieve all categories
- POST /api/categories — Create a new category (Admin only)
- GET /api/alerts — Retrieve all system alerts (Admin only)
- GET /api/alerts/active — Retrieve current critical stock alerts

7. Testing Requirements

The following testing standards must be met before project completion:

Test Type	Tool	Scope
Unit Testing	JUnit 5 + Mockito	All backend service methods must have unit test coverage.
Integration Testing	Spring Boot Test	API endpoints tested with mock HTTP requests and database.
Manual Testing	Test Case Documentation	All user-facing features tested against defined test cases.
Bug Tracking	Jira	All identified defects logged, prioritized, and tracked to resolution.